Econ 524 Midterm (50 total points) Name:
Fall 2025

1. (5 points) Assume there is a variable called x that is of type int and is a positive integer. Write a Python code fragment that computes x! (i.e. factorial of x = x*(x-1)*(x-2)*...*2*1). Save x! as a variable called y. Do not use any built-in functions (you may use len and range and math operators).

```
y=x
for i in range(1,x):
    y=y*i
```

2. (5 points) (Do not use any imports) Write a function called inches_to_feet that takes as input a positive integer which represents the number of inches. The function should return a dictionary with two keys, "feet" and "inches", that map to the number of feet and inches based on the input. For example,

inches_to_feet(66) should return a dictionary where "feet" maps to 5 and "inches" maps to 6 since 66" is 5'6".
inches_to_feet(8) should return a dictionary where "feet" maps to 0 and "inches" maps to 8.
inches_to_feet(120) should return a dictionary where "feet" maps to 10 and "inches" maps to 0.

```
def inches_to_feet(x):
    d={}
    d["feet"]=x//12
    d["inches"]=x%12
    return d
```

3. (6 points) Write a function called f that takes a list as input and returns a new list. The new list should contain the squares of all positive (or 0) valued elements in the original list. Negative values in the input list should not appear at all in the return list.
For example, f([1, 0, -1, 3, -4, 5, -6]) should return [1, 0, 9, 25].
The original list should not be modified. No imports allowed.

```
def f(x):
    r = []
    for v in x:
        if v >= 0:
            r.append(v**2)

    return r
```

4. (6 points). Suppose you have panel data for 1000 firms over 4 years in a dataframe called long that has 3 columns: id, year, and outcome. The dataframe has 4000 rows corresponding to the 4 years of data for the 1000 firms. Each firm id appears exactly 4 times in the dataframe, once for each year (years = 1, 2, 3, and 4). The four entries for each firm appear in consecutive rows ordered from year 1 to year 4, as shown in the example below. Write a code fragment to create a new dataframe called wide that has 1000 rows and 5 columns: id, year1, year2, year3, and year4. The year1, year2, etc. columns should have firm outcomes for the 4 years for each firm id. The new dataframe should have 1000 rows corresponding to 1000 unique firm ids. You may use numpy/pandas but you may not use functions that solve this problem entirely (for example, do not use pivot or related functions). One possible solution involves no functions except list.append() and pd.DataFrame(). The order of rows in the wide dataframe does not matter. Below is an example of first 8 rows of long and corresponding 2 rows in wide.

id	year	outcome
100	1	3
100	2	4
100	3	3
100	4	5
104	1	2
104	2	3
104	3	4
104	4	6
...		

Id	year1	year2	year3	year4
100	3	4	3	5
104	2	3	4	6
...				

```
ids = []
y1 = []
y2 = []
y3 = []
y4 = []
i = 0
while i < 4000:
    ids.append(long.id[i])
    y1.append(long.outcome[i])
    y2.append(long.outcome[i+1])
    y3.append(long.outcome[i+2])
    y4.append(long.outcome[i+3])
    i = i+4
wide = pd.DataFrame({"id": ids, "year1": y1, "year2": y2,
                     "year3": y3, "year4": y4})
```

5. (8 points) Insertion Sort

Insertion sort is a sorting algorithm that works as follows. Suppose x is the input list with length n.

Starting with i=1 and continuing until i=n-1 (inclusive of i=n-1).

Let j = i
Repeat as long as j>=1 AND x[j]<x[j-1]
Swap x[j] and x[j-1]
Decrease j by 1.
Increase i by 1.

Write a function called inssort that takes a single argument of type list (you may assume that list contains only float values) and returns a new list that has the values of the input list sorted in ascending order. The input list should not be modified. You should use the insertion sort algorithm described above – do not use built-in sorting functions or any built-in functions other than len, range, mathematical/logical operators, and methods of the list class.

```
def inssort(x):
    r = x.copy()
    for i in range(1, n):
        j = i
        while j>=1 and r[j]<r[j-1]:
            temp = r[j]
            r[j] = r[j-1]
            r[j-1] = temp
            j = j-1

    return r
```

The best-case scenario for this algorithm is if the list is already sorted. What is the time complexity of the best case, Ω() in terms of n, the length of the list?

$\Omega(n)$ (while loop never starts since r[j] always > r[j-1] ∴ just for loop)

Now consider a scenario where the list is sorted in reverse order (descending order). Use that scenario to determine the worst-case O() for the algorithm.

$O(n^2)$ while loop has average of $\frac{n}{2}$ iterations (nested in for loop w/ n iterations) $n \cdot \frac{n}{2} \rightarrow n^2$

6. (8 points) Consider the code fragment:

```
x=[0,1]
n=7
def f(n):
        x=[0,1]
        while n>1:
                x.append(2*n+1)
                n=n//2
        return x
y = f(n)
```

Global
x = [0, 1]
n = 7

f
n = ~~7~~ ~~3~~ 1
x = [0, 1]
[0, ~~1~~, 15]
[0, 1, 15, 7]

(a) What is y equal to after running this code?

[0, 1, 15, 7]

(b) What is x equal to after running this code?

[0, 1]

(c) What is n equal to after running this code?

7

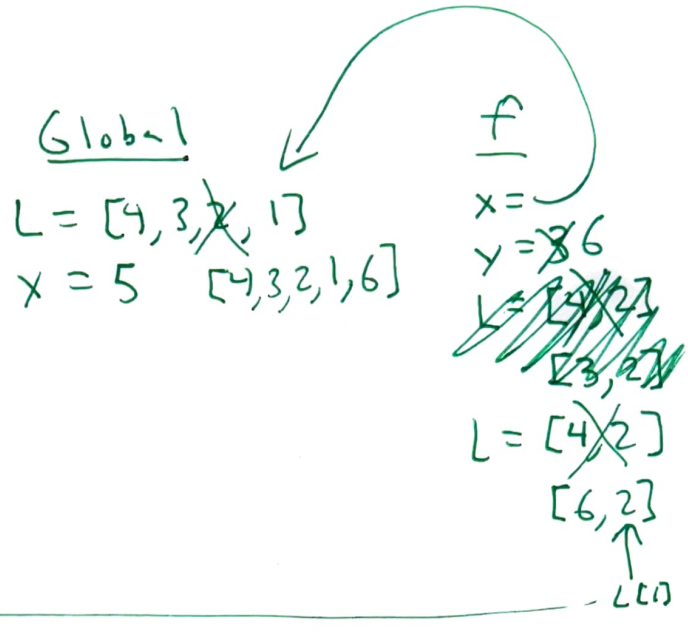
(d) What is the speed of the algorithm in the f() function in big O notation as a function of the magnitude of the input n?

O(log n)

7. (8 points) Consider the code fragment:

```
L = [4,3,2,1]
x = 5
def f(x,y):
        y = y+3
        L=x[::2]
        L[0] = y
        x.append(y)
        return L[1]
        x[1] = 8

y = f(L,L[1])*x
```



(a) What does f(L,L[1]) return?

2

(b) What is y equal to after running this code?

10 (2*5)

(c) What is x equal to after running this code?

5

(d) What is L equal to after running this code?

[4,3,2,1,6]

8. (4 points)
Below is a search_matrix function that has two inputs and returns a bool.
#m: square matrix (2-dimensional numpy array) to search
#value: value to search for
#return: TRUE if value is in matrix, FALSE otherwise

```
def search_matrix(m,value):
    n=m.shape[0]
    for i in range(n):
        for j in range(n):
            if (m[i,j]==value):
                return True
    return False
```

(a) Identify O() and Ω() for this function in terms of n, the number of rows/columns in the matrix m.

$\Omega(1)$

$O(n^2)$

Below is a search_sorted_array function that two inputs and returns a bool.
#a: a float array in ascending order
#value: value to search for
#return: TRUE if value is in array, FALSE otherwise

```
def search_sorted_array(a, value):
    while(len(a)>2):
        n=len(a)
        mid=n//2
        if(a[mid]==value):
            return True
        elif (a[mid]>value):
            a=a[0:mid]
        else:
            a=a[(mid+1):n]
    if a[0]==value:
        return True
    if len(a)>1:
        if a[1]==value:
            return True
    return False
```

(b) Identify O() and Ω() for this function in terms of n, the length of the array a.

$\Omega(1)$

$O(\log n)$